OOPs Sessional 2

Introduction to inheritance, protected access specifier, base class and derived class

Inheritance is the process of creating new classes, called *derived classes*, from existing or *base classes*. The derived class inherits all the capabilities of the base class but
can add embellishments and refinements of its own. The base class is unchanged by this
process.

Inheritance is an essential part of OOP. Its big payoff is that it permits code *reusability*. Once
a base class is written and debugged, it need not be touched again, but, using inheritance, can
nevertheless be adapted to work in different situations. Reusing existing code saves time and
money and increases a program's reliability. Inheritance can also help in the original conceptu-
alization of a programming problem, and in the overall design of the program.

See the diagrams from book for a quick revision

Protected Keyword

We don't want to make count public, since that would allow it to be accessed by any function anywhere in the program and eliminate the advantages of data hiding. A protected member, on the other hand, can be accessed by member

functions in its own class or—and here’s the key—in any class derived from its own class. It can’t be accessed from functions outside these classes, such as main(). This is just what we want. The situation is shown in Figure 9.4.

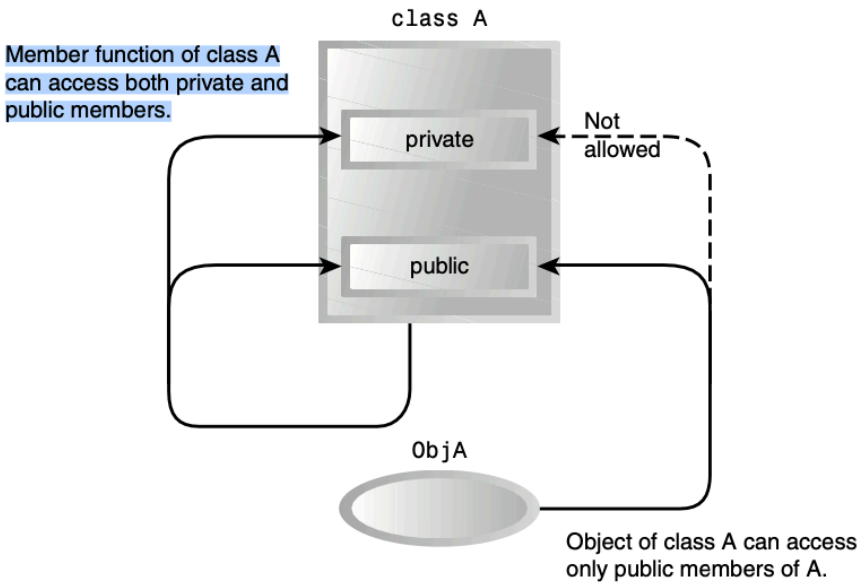

FIGURE 9.3
Access specifiers without inheritance.

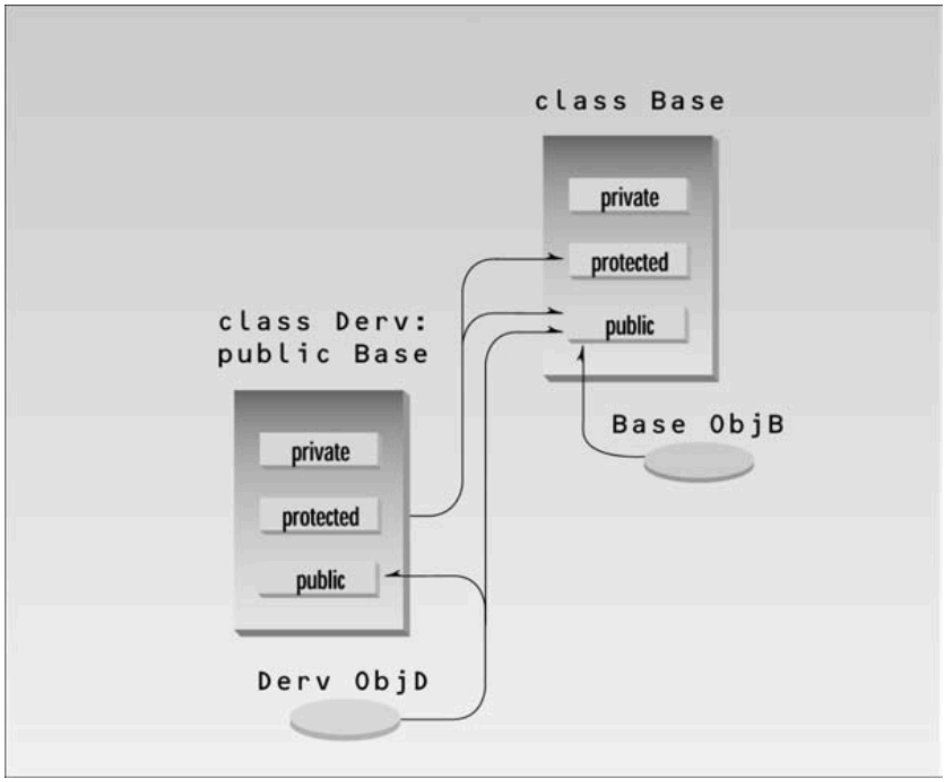

FIGURE 9.4
Access specifiers with inheritance.

The moral is that if you are writing a class that
future, as a base class for other classes, then any member data that the derived classes might
need to access should be made protected rather than private. This ensures that the class is
“inheritance ready.”

TABLE 9.1 Inheritance and Accessibility

Access Specifier	*Accessible from Own Class*	*Accessible from Derived Class*	*Accessible from Objects Outside Class*
public	yes	yes	yes
protected	yes	yes	no
private	yes	no	no

Dangers of protected

You should know that there’s a disadvantage to making class members protected. Say you’ve written a class library, which you’re distributing to the public. Any programmer who buys this library can access protected members of your classes simply by deriving other classes from them. This makes protected members considerably less secure than private members. To avoid corrupted data, it’s often safer to force derived classes to access data in the base class using only public functions in the base class, just as ordinary main() programs must do. Using the protected specifier leads to simpler programming, so we rely on it—perhaps a bit too much— in the examples in this book. You’ll need to weigh the advantages of protected against its disadvantages in your own programs.

Derived Class Constructors Pg 380 of book see this full topic here also written

There’s a potential glitch in the COUNTEN program. What happens if we want to initialize a CountDn object to a value? Can the one-argument constructor in Counter be used? The answer is no. As we saw in COUNTEN, the compiler will substitute a no-argument constructor from the base class, but it draws the line at more complex constructors. To make such a definition work we must write a new set of constructors for the derived class. This is shown in the COUNTEN2 program.

```
// counten2.cpp
// constructors in derived class
#include <iostream>
using namespace std;
////////////////////////////////////////////////////////////////
class Counter
   {
   protected:                            //NOTE: not private
      unsigned int count;                //count
   public:
      Counter() : count()                //constructor, no args
         {  }
      Counter(int c) : count(c)          //constructor, one arg
         {  }
      unsigned int get_count() const     //return count
         { return count; }
      Counter operator ++ ()             //incr count (prefix)
         { return Counter(++count); }
   };
////////////////////////////////////////////////////////////////
class CountDn : public Counter
   {
   public:
      CountDn() : Counter()              //constructor, no args
         { }
      CountDn(int c) : Counter(c)        //constructor, 1 arg
         { }
      CountDn operator -- ()             //decr count (prefix)
         { return CountDn(--count); }
   };
////////////////////////////////////////////////////////////////
int main()
   {
```

```
////////////////////////////////////////////////////////////
int main()
   {
```

```
CountDn c1;                          //class CountDn
CountDn c2(100);

cout << "\nc1=" << c1.get_count();    //display
cout << "\nc2=" << c2.get_count();    //display

++c1; ++c1; ++c1;                    //increment c1
cout << "\nc1=" << c1.get_count();    //display it

--c2; --c2;                          //decrement c2
cout << "\nc2=" << c2.get_count();    //display it

CountDn c3 = --c2;                   //create c3 from c2
cout << "\nc3=" << c3.get_count();    //display c3
cout << endl;
return 0;
}
```

This program uses two new constructors in the CountDn class. Here is the no-argument constructor:

```
CountDn() : Counter()
{}
```

This constructor has an unfamiliar feature: the function name following the colon. This construction causes the CountDn() constructor to call the Counter() constructor in the base class. In main(), when we say

```
CountDn c1;
```

the compiler will create an object of type CountDn and then call the CountDn constructor to initialize it. This constructor will in turn call the Counter constructor, which carries out the work. The CountDn() constructor could add additional statements of its own, but in this case it doesn't need to, so the function body between the braces is empty.

Calling a constructor from the initialization list may seem odd, but it makes sense. You want to initialize any variables, whether they're in the derived class or the base class, before any statements in either the derived or base-class constructors are executed. By calling the base-class constructor before the derived-class constructor starts to execute, we accomplish this.

The statement

```
CountDn c2(100);
```

in main() uses the one-argument constructor in CountDn. This constructor also calls the

corresponding one-argument constructor in the base class:

```
CountDn(int c) : Counter(c) ← argument c is passed to Counter
{}
```

This construction causes the argument c to be passed from CountDn() to Counter(), where it

is used to initialize the object.

In main(), after initializing the c1 and c2 objects, we increment one and decrement the other
and then print the results. The one-argument constructor is also used in an assignment statement.

```
CountDn c3 = --c2;
```

Overriding Member Functions

You can use member functions in a derived class that override—that is, have the same name as—those in the base class. You might want to do this so that calls in your program work the same way for objects of both base and derived classes.

“Abstract” Base Class

Notice that we don’t define any objects of the base class employee. We use this as a general
class whose sole purpose is to act as a base from which other classes are derived.

The laborer class operates identically to the employee class, since it contains no additional
data or functions. It may seem that the laborer class is unnecessary, but by making it a separate
class we emphasize that all classes are descended from the same source, employee. Also, if in
the future we decided to modify the laborer class, we would not need to change the declaration
for employee.

Classes used only for deriving other classes, as employee is in EMPLOY, are sometimes loosely called *abstract classes*, meaning that no actual instances (objects) of this class are created.

Constructors and Member Functions

There are no constructors in either the base or derived classes, so the compiler creates objects
of the various classes automatically when it encounters definitions like

manager m1, m2;
using the default constructor for manager calling the default constructor for employee.

The getdata() and putdata() functions in employee accept a name and number from the
user and display a name and number. Functions also called getdata() and putdata() in the
manager and scientist classes use the functions in employee, and also do their own work.
In manager, the getdata() function asks the user for a title and the amount of golf club dues,
and putdata() displays these values. In scientist, these functions handle the number of
publications.

Public and Private Inheritance

C++ provides a wealth of ways to fine-tune access to class members. One such access-control
mechanism is the way derived classes are declared. Our examples so far have used publicly
derived classes, with declarations like

```
class manager : public employee
```

which appeared in the EMPLOY example.

What is the effect of the public keyword in this statement, and what are the alternatives?
Listen up: The keyword public specifies that objects of the derived class are able to access
public member functions of the base class. The alternative is the keyword private. When this
keyword is used, objects of the derived class cannot access public member functions of the
base class. Since objects can never access private or protected members of a class, the result
is that no member of the base class is accessible to objects of the derived class.

Access Combinations

See this full topic from book Book page Number - 397

If you don't supply any access specifier when creating a class, private is assumed.

Multiple Inheritance

A class can be derived from more than one base class. This is called *multiple inheritance*.

Figure 9.9 shows how this looks when a class C is derived from base classes A and B.

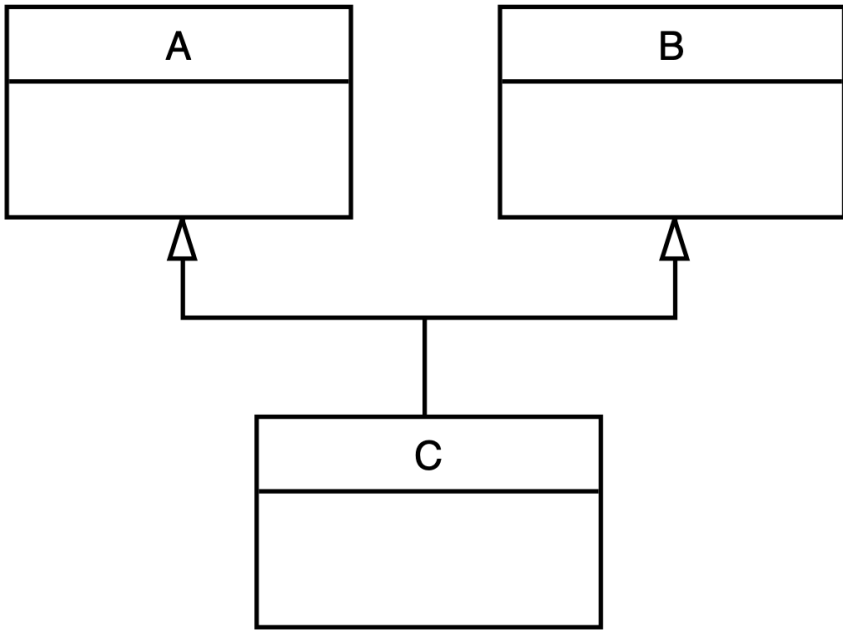

FIGURE 9.9
UML class diagram for multiple inheritance.

The syntax for multiple inheritance is similar to that for single inheritance. In the situation
shown in Figure 9.9, the relationship is expressed like this:

9

class A `// base class A`
{

};
class B `// base class B`

{

```
   };
class C : public A, public B.   // C is derived from A and B
```

{
};

The base classes from which C is derived are listed following the colon in C's specification;
they are separated by commas.

Summary

A class, called the *derived class*, can inherit the features of another class, called the *base class*.
The derived class can add other features of its own, so it becomes a specialized version of the
base class. Inheritance provides a powerful way to extend the capabilities of existing classes,
and to design programs using hierarchical relationships.

9

Accessibility of base class members from derived classes and from objects of derived classes is
an important issue. Data or functions in the base class that are prefaced by the keyword *pro-*
tected can be accessed from derived classes but not by any other objects, including objects of
derived classes. Classes may be publicly or privately derived from base classes. Objects of a
publicly derived class can access public members of the base class, while objects of a privately
derived class cannot.

A class can be derived from more than one base class. This is called *multiple inheritance*. A
class can also be contained within another class.

Inheritance permits the reusability of software: Derived classes can extend the capabilities of base classes with no need to modify—or even access the source code of—the base class. This leads to new flexibility in the software development process, and to a wider range of roles for software developers.

Pointers to objects, Virtual functions, run time polymorphism, Pointers inside classes, dynamic allocation for objects, new and delete, virtual destructors

Pointer to objects Pg 464 of the book

Pointers can point to objects as well as to simple data types and arrays. We've seen many
examples of objects defined and given a name, in statements like

```
Distance dist;
```

where an object called dist is defined to be of the Distance class.

Sometimes, however, we don't know, at the time that we write the program, how many objects
we want to create. When this is the case we can use new to create objects while the program is

running. As we've seen, new returns a pointer to an unnamed object. Let's look at a short exam-
ple program, ENGLPTR, that compares the two approaches to creating objects.

```
// englptr.cpp
// accessing member functions by pointer
#include <iostream>
using namespace std;
////////////////////////////////////////////////////////////////
class Distance
   {



    private:
       int feet;



       float inches;
   public:



       void getdist()
         {
//English Distance class



//get length from user
          cout << "\nEnter feet: ";  cin >> feet;
         cout << "Enter inches: ";  cin >> inches;
         }



       void showdist()       //display distance
         { cout << feet << "\'-" << inches << '\"'; }



    };
////////////////////////////////////////////////////////////////
int main()
{
Distance dist;
dist.getdist();
```

```
dist.showdist();
```

```
Distance* distptr;
distptr = new Distance;  //points to new Distance object
distptr->getdist();      //access object members
distptr->showdist();     //   with -> operator
cout << endl;
return 0;
}
```

```
//define a named Distance object
//access object members
//   with dot operator
```

```
//pointer to Distance
```

Pointers

This program uses a variation of the English Distance class seen in previous chapters. The
main() function defines dist, uses the Distance member function getdist() to get a distance
from the user, and then uses showdist() to display it.

Referring to Members

ENGLPTR then creates another object of type Distance using the new operator, and returns a
pointer to it called distptr.

The question is, how do we refer to the member functions in the object pointed to by distptr?
You might guess that we would use the dot (.) membership-access operator, as in

```
distptr.getdist();   // won't work; distptr is not a variable
```

but this won't work. The dot operator requires the identifier on its left to be a variable. Since
distptr is a pointer to a variable, we need another syntax. One approach is to *dereference* (get
the contents of the variable pointed to by) the pointer:

```
(*distptr).getdist();  // ok but inelegant
```

However, this is slightly cumbersome because of the parentheses. (The parentheses are neces-
sary because the dot operator (.) has higher precedence than the dereference operator (*). An
equivalent but more concise approach is furnished by the membership-access operator, which
consists of a hyphen and a greater-than sign:

distptr→getdist(); // better approach

As you can see in ENGLPTR, the → operator works with pointers to objects in just the same way

that the . operator works with objects. Here's the output of the program:

Enter feet: 10 ← this object uses the dot operator
Enter inches: 6.25
10′-6.25″

Enter feet: 6 ← this object uses the → operator
Enter inches: 4.75
6′-4.75″

Another approach to new

`int main()`

```
  {
  Distance& dist = *(new Distance);  // create Distance object
```

```
    dist.getdist();
    dist.showdist();
```

```
    cout << endl;
    return 0;
```

}

The expression

```
// alias is “dist”
// access object members
//    with dot operator
```

new Distance
returns a pointer to a memory area large enough for a Distance object, so we can refer to the

original object as

```
*(new Distance)
```

This is the object pointed to by the pointer. Using a reference, we define dist to be an object
of type Distance, and we set it equal to *(new Distance). Now we can refer to members of
dist using the dot membership operator, rather than →.

This approach is less common than using pointers to objects obtained with new, or simply
declaring an object, but it works in a similar way.

Summary about pointers

The new operator obtains a specified amount of memory from the system and returns a pointer to the memory. This operator is used to create variables and data structures during program execution. The delete operator releases memory obtained with new.

When a pointer points to an object, members of the object's class can be accessed using the access operator →. The same syntax is used to access structure members.
Classes and structures may contain data members that are pointers to their own type. This per- mits the creation of complex data structures such as linked lists. There can be pointers to pointers. These variables are defined using the double asterisk; for example, int** pptr.

Virtual Functions

Virtual means existing in appearance but not in reality.

When virtual functions are used, a program that appears to be calling a function of one class may in reality be calling a function of a different class. Why are virtual functions needed? Suppose you have a number of objects of different classes but you want to put them all in an array and perform a particular operation on them using the same function call. For example, suppose a graphics program includes several different shapes: a triangle, a ball, a square, and so on, as in the MULTSHAP program in Chapter9, "Inheritance." Each of these classes has a member function draw() that causes the object to be drawn on the screen.

Now suppose you plan to make a picture by grouping a number of these elements together, and
you want to draw the picture in a convenient way. One approach is to create an array that holds
pointers to all the different objects in the picture. The array might be defined like this:

```
shape* ptrarr[100];  // array of 100 pointers to shapes
```

If you insert pointers to all the shapes into this array, you can then draw an entire picture using
a simple loop:

```
for(int j=0; j<N; j++)
```

```
    ptrarr[j]->draw();
```

This is an amazing capability: Completely different functions are executed by the same func-
tion call. If the pointer in ptrarr points to a ball, the function that draws a ball is called; if it
points to a triangle, the triangle-drawing function is called. This is called *polymorphism*, which
means *different forms*. The functions have the same appearance, the draw() expression, but dif-
ferent actual functions are called, depending on the contents of ptrarr[j]. Polymorphism is
one of the key features of object-oriented programming, after classes and inheritance.

For the polymorphic approach to work, several conditions must be met. First, all the different
classes of shapes, such as balls and triangles, must be descended from a single base class
(called shape in MULTSHAP). Second, the draw() function must be declared to be virtual in
the base class

💡 At runtime, when it is known what class is pointed to by ptr, the appropriate version of function will be called. This is called *late binding or dynamic binding.*
(Choosing functions in the normal way, during compilation, is called *early binding* or *static binding*.) Late binding requires some overhead but provides increased power and
flexibility.

Abstract Classes and Pure Virtual Functions

Think of the shape class in the multshap program in Chapter 9. We'll never make an object of
the shape class; we'll only make specific shapes such as circles and triangles. When we will
never want to instantiate objects of a base class, we call it an *abstract class*. Such a class exists
only to act as a parent of derived classes that will be used to instantiate objects. It may also
provide an interface for the class hierarchy.

How can we make it clear to someone using our family of classes that we don't want anyone to
instantiate objects of the base class? We could just say this in the documentation, and count on
the users of the class to remember it, but of course it's much better to write our classes so that
such instantiation is impossible. How can we can do that? By placing at least one *pure virtual*
function in the base class. A pure virtual function is one with the expression =0 added to the
declaration. This is shown in the VIRTPURE example.

```
// virtpure.cpp
// pure virtual function
#include <iostream>
using namespace std;
////////////////////////////////////////////////////////////////
class Base                        //base class
```

{
public:

```
      virtual void show() = 0;    //pure virtual function
   };
```

```
////////////////////////////////////////////////////////////////
```

```
class Derv1 : public Base
  {
```

```
   public:
     void show()
```

```
//derived class 1
```

```
         { cout << “Derv1\n”; }
////////////////////////////////////////////////////////////////
```

};

```
class Derv2 : public Base
   {
```

```
   public:
      void show()
```

```
         { cout << “Derv2\n”; }
```

```
//derived class 2
```

```
   };
////////////////////////////////////////////////////////////////
int main()
```

{

// Base bad;

```
   Base* arr[2];
   Derv1 dv1;
```

```
//can’t make object from abstract class
//array of pointers to base class
//object of derived class 1
```

510

511

VIRTUALFUNCTIONS

Derv2 dv2;

```
arr[0] = &dv1;
arr[1] = &dv2;
```

```
arr[0]->show();
arr[1]->show();
return 0;
}
```

`//object of derived class 2`

`//put address of dv1 in array`
`//put address of dv2 in array`

`//execute show() in both objects`

11

Here the virtual function show() is declared as

```
virtual void show() = 0;  // pure virtual function
```

The equal sign here has nothing to do with assignment; the value 0 is not assigned to anything.
The =0 syntax is simply how we tell the compiler that a virtual function will be pure. Now if in
main() you attempt to create objects of class Base, the compiler will complain that you’re try- ing to instantiate an object of an abstract class. It will also tell you the name of the pure virtual function that makes it an abstract class. Notice that, although this is only a declaration, you never need to write a definition of the base class show(), although you can if you need to.
Once you’ve placed a pure virtual function in the base class, you must override it

in all the derived classes from which you want to instantiate objects. If a class doesn't override the pure virtual function, it becomes an abstract class itself, and you can't instantiate objects from it (although you might from classes derived from it). For consistency, you may want to make all the virtual functions in the base class pure.

As you can see, we've made another, unrelated, change in VIRTPURE: The addresses of the member functions are stored in an array of pointers and accessed using array elements. This works in just the same way as using a single pointer. The output of VIRTPURE is the same as VIRT:

Derv1 Derv2

> **Inheritance readiness -** A class should be written in such a way that it is ready for inheritance every time so that some other class can be derived from it like making data members as protected and making destructors virtual

Virtual Destructors

Base class destructors should always be virtual. Suppose you use delete with a base class
pointer to a derived class object to destroy the derived-class object. If the base-class destructor
is not virtual then delete, like a normal member function, calls the destructor for the base
class, not the destructor for the derived class. This will cause only the base part of the object to
be destroyed.

Summary

Virtual functions provide a way for a program to decide while it is running what function to
call. Ordinarily such decisions are made at compile time. Virtual functions make possible
greater flexibility in performing the same kind of action on different kinds of objects. In partic-
ular, they allow the use of functions called from an array of type pointer-to-base that actually
holds pointers (or references) to a variety of derived types. This is an example of *polymor-*
phism. Typically a function is declared virtual in the base class, and other functions with the
same name are declared in derived classes.

The use of one or more pure virtual functions in a class makes the class *abstract*, which means
that no objects can be instantiated from it.